

Abstract Methods and Abstract Classes (Java)

Corrected study notes | CE = Compile-time Error

1. Basic definitions

- **Abstract method:** a method declared with the **abstract** keyword and **no body**; it ends with a semicolon. Example: `abstract void m1();`
- **Abstract class:** a class declared with the **abstract** keyword. If a class contains even one abstract method, the class **must** be declared abstract.
- Abstract methods (prototypes) are defined by the **super class developer**; the **sub class developer** provides the body and logic according to business requirements.

Correction: Your notes said "a method that does not have a body is called an abstract method". A method without a body is only legal if it is marked **abstract** (or **native**, or is an interface method). Without the keyword it is a CE (Rule 1).

Q) Why must a bodiless method have the abstract keyword?

In a class, methods are normally defined **with** a body. Removing the body changes this default property, so it must be announced with the **abstract** keyword in the method prototype.

2. The 4 rules

Rule 1: A method without a body must be declared abstract, otherwise CE.

```
class Example {  
    void m1();           // CE: missing method body, or declare abstract  
}
```

Rule 2: A class with an abstract method must be declared abstract, otherwise CE.

```
class Example {           // CE: Example is not abstract and does not  
    abstract void m1();    //      override abstract method m1() in Example  
}  
  
abstract class Example {  // OK  
    abstract void m1();  
}
```

Rule 3: An abstract class cannot be instantiated, otherwise CE.

```
abstract class Example {  
    abstract void m1();  
    public static void main(String[] args) {  
        Example e = new Example(); // CE: Example is abstract; cannot be instantiated  
    }  
}
```

Rule 4: A sub class of an abstract class must override **all** abstract methods, or itself be declared abstract, otherwise CE.

```

abstract class Example {
    abstract void m1();
    abstract void m2();
}
class Sample extends Example {      // CE: Sample is not abstract and does not
    void m1() { System.out.println("m1"); } // override abstract method m2()
}

// Solution 1: declare the sub class abstract
abstract class Sample extends Example {
    void m1() { System.out.println("m1"); }
}
// Solution 2: override both methods
class Sample extends Example {
    void m1() { System.out.println("m1"); }
    void m2() { System.out.println("m2"); }
}

```

Q) Why can't an abstract class be instantiated?

It may contain abstract methods that have no body. If the compiler allowed `new Example()`, we could call an abstract method through that object and the JVM would have nothing to execute at runtime. So the compiler blocks instantiation.

Correction: Your notes said "because it is not fully implemented". That is true for abstract classes that have abstract methods, but a **concrete class declared abstract** (section 7) is fully implemented and still cannot be instantiated. The rule applies to every class marked `abstract`.

3. Members of an abstract class

Q) What members can an abstract class have?

Everything a normal class can have, plus abstract methods:

#	Member	#	Member
1	Static variable	5	Static method
2	Non-static (instance) variable	6	Non-static method
3	Static block	7	Constructor
4	Non-static (instance) block	8	Abstract method

Correction: Your notes said "9 types". Counting the **main()** method as a 9th makes the number work, but `main` is just a static method. The distinct kinds are the 8 above (`main`, nested classes and final methods are also allowed).

Q) Is memory allocated for the abstract class when a sub class object is created?

Yes. Its non-static members get memory when an object of a concrete sub class is created, and the abstract class constructor runs (through `super()`), but no separate object of the abstract class exists.

Q) How do we run static and non-static concrete members of an abstract class?

- **Static members:** directly from a `main` method (using the class name, or directly from a sub class).
- **Non-static members:** only through an object of a **concrete sub class**.

Diagram from your screenshot

```
abstract class Example {  
    static void m1() { System.out.println("Example m1"); }  
    void m2()      { System.out.println("Example m2"); }  
}
```

Calling from a normal class (does not extend Example)	Result
Example.m1();	OK (static method, called with class name)
Example e = new Example();	CE (abstract class cannot be instantiated)
e.m2();	CE (no object could be created)
Calling from a sub class (Sample extends Example)	Result
m1();	OK (static method inherited)
Sample e = new Sample();	OK (Sample is concrete)
e.m2();	OK (non-static method via sub class object)

4. Practice program 1: output?

```
abstract class Example {  
    abstract void m1();  
    static int a = 10;    int x = 20;  
    static { System.out.println("Example SB"); }  
    { System.out.println("Example NSB"); }  
    Example() { System.out.println("Example Constructor"); }  
    static void m2() { System.out.println("Example SM"); }  
    void m3() { System.out.println("Example NSM"); }  
    public static void main(String[] args) {  
        System.out.println("Example main");  
        System.out.println("a:" + a);  
        m2();  
        Example e = new Example();    // <-- problem  
        e.m3();  
    }  
}
```

Answer: compile-time error at new Example() - Example is abstract; cannot be instantiated. Nothing runs. If you delete the last two lines of main, the output is: Example SB, Example main, a:10, Example SM. Static members can be executed from main, but non-static members need a concrete sub class object (program 2).

5. Practice program 2: running non-static members via a sub class

```
class Sample extends Example {
    static int b = 30;    int y = 40;
    static { System.out.println("Sample SB"); }
    { System.out.println("Sample NSB"); }
    Sample() { System.out.println("Sample Constructor"); }
    void m1() { System.out.println("m1 in Sample"); }
    static void m4() { System.out.println("Sample SM"); }
    void m5() { System.out.println("Sample NSM"); }
    public static void main(String[] args) {
        System.out.println("\nSample main");
        System.out.println("a: " + a);
        System.out.println("b: " + b);
        m2(); m4();
        System.out.println();
        Sample s = new Sample();
        s.m1(); s.m3(); s.m5();
    }
}
```

Output when you run `java Sample` (Example.main is not run):

```
Example SB          <- super class static block first (class loading)
Sample SB           <- then sub class static block

Sample main
a: 10
b: 30
Example SM          <- m2()
Sample SM           <- m4()
                   <- blank line
Example NSB         <- object creation: super non-static block
Example Constructor <- super constructor
Sample NSB          <- sub non-static block
Sample Constructor  <- sub constructor
m1 in Sample
Example NSM         <- m3()
Sample NSM          <- m5()
```

Note: Order rule: static blocks (super, then sub) → main → on new: super non-static block, super constructor, sub non-static block, sub constructor.

6. Modifiers with abstract

Q) Can an abstract method be static?

No. A static method can be called directly with the class name without an object, but an abstract method has no body for the JVM to run. CE: *illegal combination of modifiers: abstract and static*.

Q) Can an abstract method be final?

No. An abstract method must be overridden in the sub class, and final forbids overriding. CE: *illegal combination of modifiers: abstract and final*.

Q) Can an abstract method be private?

No. It must be inherited (visible) in the sub class so that it can be overridden, and private members are not inherited. CE: *illegal combination of modifiers: abstract and private*.

```
abstract class Example {
    static abstract void m1();    // CE
    final abstract void m1();    // CE
    private abstract void m1();  // CE
}
```

Q) Which modifiers are legal with abstract?

Only **public** and **protected** (access modifiers), or **no access modifier** (default/package access). Combining abstract with private, static, final, native, synchronized or strictfp gives CE: illegal combination of modifiers. (transient and volatile are for variables only, so they are not allowed on any method.) Repeating the keyword, as in `abstract abstract void m1();`, gives CE: repeated modifier.

Correction: Your notes say "only two modifiers" but forgot to mention that **default access** (writing no modifier, as in `abstract void m1();`) is also legal. Also, transient and volatile are for fields only, so they are illegal on any method.

Q) Identify the valid abstract method declarations

```
abstract class Example {
    abstract void m1();           // VALID
    abstract int m2();           // VALID
    static abstract void m3();    // INVALID (abstract + static)
    final abstract void m4();     // INVALID (abstract + final)
    private abstract void m5();   // INVALID (abstract + private)
    protected abstract void m6(); // VALID
    public abstract void m7();    // VALID
}
```

Valid: m1, m2, m6, m7.

7. Concrete class declared as abstract

Q) Can we declare a concrete class (no abstract methods) as abstract?

Yes. It is a way to prevent anyone from instantiating a class that is meant to be extended first.

Why do it? To make sure the class's non-static members are accessible only through a **sub class** object. Its static members can still be used with the class name.

Predefined examples: `java.awt.Component`, `javax.servlet.http.HttpServlet` (abstract classes with no abstract methods in HttpServlet's case).

8. Quick revision

- Abstract method = **abstract** keyword + no body + semicolon.
- Class with an abstract method must be abstract; an abstract class cannot be instantiated (CE).
- Sub class must implement all abstract methods, or be abstract itself.
- Abstract class can have constructors, static/non-static blocks, variables, and methods; constructors run via `super ( )` when a sub class object is created.
- `abstract` cannot combine with `private`, `static`, `final`, `native`, `synchronized`, `strictfp`. Allowed: `public`, `protected`, default access.
- A concrete class can be declared abstract to stop direct instantiation.